© copyright : spiraltrain@gmail.com

Exercise : Inheritance

Step 3 : Inheritance

Derive a `WageEmployee` class from the `Employee` class. The `WageEmployee` class inherits the attributes and public methods of the `Employee` class. Define two additional attributes in `WageEmployee` `wage`, representing the hourly `wage`, and `hours`, representing the hours worked. Both these attributes should be private. Also implement get, set and del methods for these attributes. Your code could look as follows :

```
    def get_wage(self):
        return self.__wage
    def get_hours(self):
        return self.__hours
    def set_wage(self, value):
        self.__wage = value
    def set_hours(self, value):
        self.__hours = value
    def del_wage(self):
        del self.__wage
    def del_hours(self):
        del self.__hours
```

Put the `WageEmployee` in the same package as the `Employee`.

Further define a `calcSal` method in `WageEmployee` which returns the salary of a `WageEmployee` by multiplicating `hours` and `wage`. Your code could look as follows :

```
    def calcSal(self):
        return self.__wage * self.__hours
```

De relationship between a `WageEmployee` and an `Employee` is a generalization / specialization, also called an inheritance relation. A `WageEmployee` is a special type of `Employee`.

Add a constructor to the `WageEmployee` class which accepts type the name and id attributes. Use this constructor also to set the hours and wage attributes to default values. Notice that in order to fill the name and id attribute an appropriate `Employee` constructor must be called. It is not possible to access the attributes directly. This constructor should already be present in the `Employee` class. Call this constructor from the `WageEmployee` constructor. Use code like the following :

```
class WageEmployee(Employee) :
    def __init__(self, name, wage, hours):
        super(WageEmployee,self).__init__(name)
        print ("Calling WageEmployee constructor")
        self.__wage = wage
```

```
        self.__hours = hours
```

Also make `wage` and `hours` properties as follows :

```
    wage = property(get_wage, set_wage, del_wage, "wage's docstring")
    hours = property(get_hours, set_hours, del_hours, "hours's docstring")
```

Next declare an instance of a `WageEmployee` in the simple program which also declares the `Company` object and call its methods. Or do this in the script if you did not create the `Company` class. Try to use the private name and id attributes from the `WageEmployee` class. You will not succeed. Use the accessor methods instead. Your code could look like the following :

```
w1 = employees.wageemployee.WageEmployee("Werner Heisenberg", 10, 100)
print (w1.name)
print (w1.wage)
print (w1.hours)
```

Add a `ManagerEmployee` to the `Employee` hierarchy which is derived `Employee` and whose salary is a fixed value per month. Add a private `monthlySalary` attribute of the integer type to the `ManagerEmployee` class. Its `calcSal` method should return the value of that attribute. It should be possible to give the `ManagerEmployee` a salary in its constructor. Add a constructor and add accessor functions for the `monthlySalary` attribute. Do something like the following :

```
class ManagerEmployee(Employee) :
    def __init__(self, name, weeksal):
        super(ManagerEmployee,self).__init__(name)
        print ("Calling ManagerEmployee constructor")
        self.__weeksal = weeksal
```

Put the `ManagerEmployee` in the same package as the `Employee`.

Add a `ProgrammerEmployee` to the `Employee` hierarchy which is derived from `WageEmployee` and whose salary consists of the `WageEmployee` salary + a percentage of the lines produced, minus a percentage of the bugs introduced. Add attributes `lines` and `bugs`

Add a constructor and add accessor functions for the `lines` and `bugs` attributes.

Conceive an appropriate implementation of its `calcSal` method

Put the `ProgrammerEmployee` in the same package as the `Employee`.

Also add destructors for all these classes.

Step 4 : Polymorphism

Create instances of each of the derived `Employee` classes in the script which declares the `Company` object or in the script without the `Company` object.

Give each employee in the hierarchy, a name, id, and a salary and display these. Next add all the employees to the `employeeList` in the `Company` object. Make a function `payroll` in the `Company` object which iterates through the list of employees, calculates each of their salaries, adds them and finally returns the total payroll of the `Company`.

Calling the `calcsal` method on the employees in the list, while executing different versions of these calcsal method depending on the concrete derived `Employee` in the list, is a demonstration of the object oriented principle of polymorphism.

In Python it is sufficient for all classes to have the same method to enable polymorphism. In Java or C++ the `calcSal()` method must be present in the base class.

If time permits :

Add code that limits the permitted values for the ID attribute to values between 1 and 1000. If someone tries to set the value higher or lower than these limits an error message should be displayed or an exception should be raised and the ID value should be set to the nearest permitted value. The check can be done in the set method and the constructor.

Exercise : Decorators

Step 1

Create an Eclipse Project named `exercise5` and a Python module `calculator5.py`. Copy the contents of the `calculator4.py` from the previous exercise to `calculator5.py`. Also copy the calculations package. You will use the solution code of the previous exercise as a starting point.

Step 2 : benchmark decorator

Now create the following benchmark decorator function in the `calculations.py` :

```
def benchmark(func):
    """
    A decorator that prints the time a function takes
    to execute.
    """
    import time
    def wrapper(*args, **kwargs):
        t = time.clock()
        res = func(*args, **kwargs)
        print (func.__name__, time.clock()-t)
        return res
    return wrapper
```

Add this decorator to the functions in `calculations.py` with the following annotation :

```
@benchmark
```

Run the code and observe the result.

Step 3 : logging decorator

Now create the following logging decorator function in the `calculations.py` :

```
def logging(func):
    """
    A decorator that logs the activity of the script.
    (it actually just prints it, but it could be logging!)
    """
    def wrapper(*args, **kwargs):
        res = func(*args, **kwargs)
        print (func.__name__, args, kwargs)
        return res
    return wrapper
```

Also add this decorator to the functions in `calculations.py` with the following annotation :

```
@logging
```

Run the code and observe the result.

Step 4 : counter decorator

Finally create the following counter decorator function in the `calculations.py` :

```
def counter(func):
    """
    A decorator that counts and prints the number of times a function has
been executed
    """
    def wrapper(*args, **kwargs):
        wrapper.count = wrapper.count + 1
        res = func(*args, **kwargs)
        print ("{0} has been used: {1}x".format(func.__name__,
wrapper.count))
        return res
    wrapper.count = 0
    return wrapper
```

Also add this decorator to the functions in `calculations.py` with the following annotation :

```
@counter
```

Run the code and observe the result.

Exercise : Python and XML

Step 1 : minidom

Create a Python script that creates an XML structure with the xml minidom library looking as follows :

```
<?xml version="1.0" ?>
<employees>
  <employee type="emp">
    <name>
      Albert Einstein
    </name>
    <id>
      11111
    </id>
    <ssn>
      123456789
    </ssn>
    <age>
      55
    </age>
    <hiringDate>
      11-12-1953
    </hiringDate>
  </employee>
  <employee type="wageemp">
    <wage>
      10
    </wage>
    <hours>
      10
    </hours>
  </employee>
  <employee type="manageremp">
    <weeksal>
      10
    </weeksal>
  </employee>
  <employee type="programmeremp">
    <lines>
      10
    </lines>
    <bugs>
      10
    </bugs>
  </employee>
</employees>
```

Step 2 : Sax

Use the xml.sax library to parse and display the elements of an XML file.

Exercise : Python Libraries

You will find documentation on the use of the Python libraries in the root directory of the courseCD.

Step 1

Make a program that waits for user input and that reads a licence plate for a car with the input function. A licence plate of a car should have the following format : 2 uppercase character, a dash, 2 numbers , a dash and two uppercase characters again like in : AB-67-NK. Verify that the license plate is correct with a regular expression.